Haliç University

ACP project

Developing a RESTful API with FastAPI
IoT Early Warning System: Bosphorus Case Study

Student Number: 22092090067
Student Name: Oğuzhan Çelik
Student Section: Section 1
Teacher Name: Dr. Mohammed Al-Hubaishi

Dataset Link: https://www.kaggle.com/datasets/caetanoranieri/water-level-identification-with-lidar
Video Link: https://youtu.be/qmpfvGtDZz4

Project Overview

In this project, you will explore a real-world dataset that combines networking technology (wired/wireless networks, sensors, or IoT) with practical applications. You will perform data analysis, develop machine learning models, and create a functional FastAPI to demonstrate your understanding.

Project Steps

1. Define the application requirements and use feature engineering techniques to analyze your chosen dataset.
2. Develop ML models that outline the main idea and overall structure of your project. Compare 4 ML models to evaluate the results.
3. Test the model using FastAPI interface.
4. Ensure your project demonstrates novelty and is not similar to any previously published work.

Contents

1 Dataset Information

Name of the Dataset: Water level identification with distance sensors
Source: https://www.kaggle.com/datasets/caetanoranieri/water-level-identification-with-lidar

2 Dataset Description

The dataset contains sensor data collected in a controlled laboratory setting. The target variable is `water_level`. The inputs are LiDAR (IR), Ultrasonic (US) distance sensors, and IMU (Accelerometer/Gyroscope) data. Additionally, variables such as water turbidity, sensor angle, and distance have been controlled.

What the dataset contains: The dataset consists of sensor readings collected to measure water levels under different conditions. It includes three separate CSV files representing three different levels of water turbidity: High, Medium, and Low. Each file contains the following columns (features):

1. **ID:** A sequential identifier for each data point.
2. **IR Data:** `ir_value` and `ir_strength` from the LiDAR/Infrared sensor.
3. **Ultrasonic Data:** `us_value` measuring distance using sound waves.
4. **IMU Data:** `acc_x`, `acc_y`, `acc_z` (Accelerometer) and `gyr_x`, `gyr_y`, `gyr_z` (Gyroscope) readings to detect movement or tilt.
5. **Derived/Other:** `gyr_acc_x`, `gyr_acc_y`, `gyr_acc_z`, and `angle`.
6. **Target Variable:** `water_level`, which is the actual water level we are trying to predict or monitor.

Format of the data: The data is formatted as CSV (Comma-Separated Values). It is structured tabular data where the first row is the header containing column names, and subsequent rows contain numerical sensor readings separated by commas.

Size of the dataset: The dataset is split into three files. Each file contains 15 columns. Based on the IDs visible in the files, each file contains approximately 10,500 rows of data. Consequently, the total dataset size is roughly 31,500 records across all three turbidity levels.

Any other relevant details: The data appears to be collected in a controlled environment where the water level is changed systematically. The inclusion of IMU data suggests the system might be designed to account for sensor tilt or vibration, which is critical for accurate readings in real-world floating sensor buoys.

3 Relation to Networking Technology and Other Applications

3.1 Relation to Networking Technology (IoT & Sensors)

This dataset is a perfect example of the "Perception Layer" in an Internet of Things (IoT) architecture.

Sensors as Nodes: The LiDAR, Ultrasonic, and IMU sensors act as edge nodes that collect raw physical data. In a networking context, these sensors would interface with a microcontroller.

Wireless Transmission: To make this system useful remotely, the data wouldn't just stay on a CSV file; it would be transmitted over a network. This relates to Wireless Sensor Networks (WSNs) using protocols like Wi-Fi, LoRaWAN, or Zigbee to send the water level status to a cloud dashboard or a local server.

Edge Computing: The `water_level` target variable suggests we can train a Machine Learning model. In advanced networking, this model could run directly on the edge device (TinyML) to send alerts only when the water level is critical, saving network bandwidth.

3.2 Dataset Applications

Smart Water Management Systems: The primary application is automating water tanks in residential or agricultural settings. The system can turn pumps on or off based on the `water_level` readings, and use the turbidity data to decide if the water needs filtering.

Flood Detection and Early Warning: By deploying these sensors in riverbeds or city drainage systems, the setup can monitor rising water levels in real-time. The networking component would send an immediate SMS or app alert to authorities if the level exceeds a safety threshold.

Industrial Liquid Processing: In factories handling liquids (like beverages or chemicals), determining the exact level of liquid inside opaque tanks is crucial. The combination of Ultrasonic and IR sensors allows for redundancy—if one sensor fails or is affected by foam/steam, the other can compensate.

4 Initial Thoughts on Potential Analyses

The proposed system architecture follows a four-stage pipeline designed to process sensor data and deploy a predictive model. The overall structure is visualized below in Figure 1.

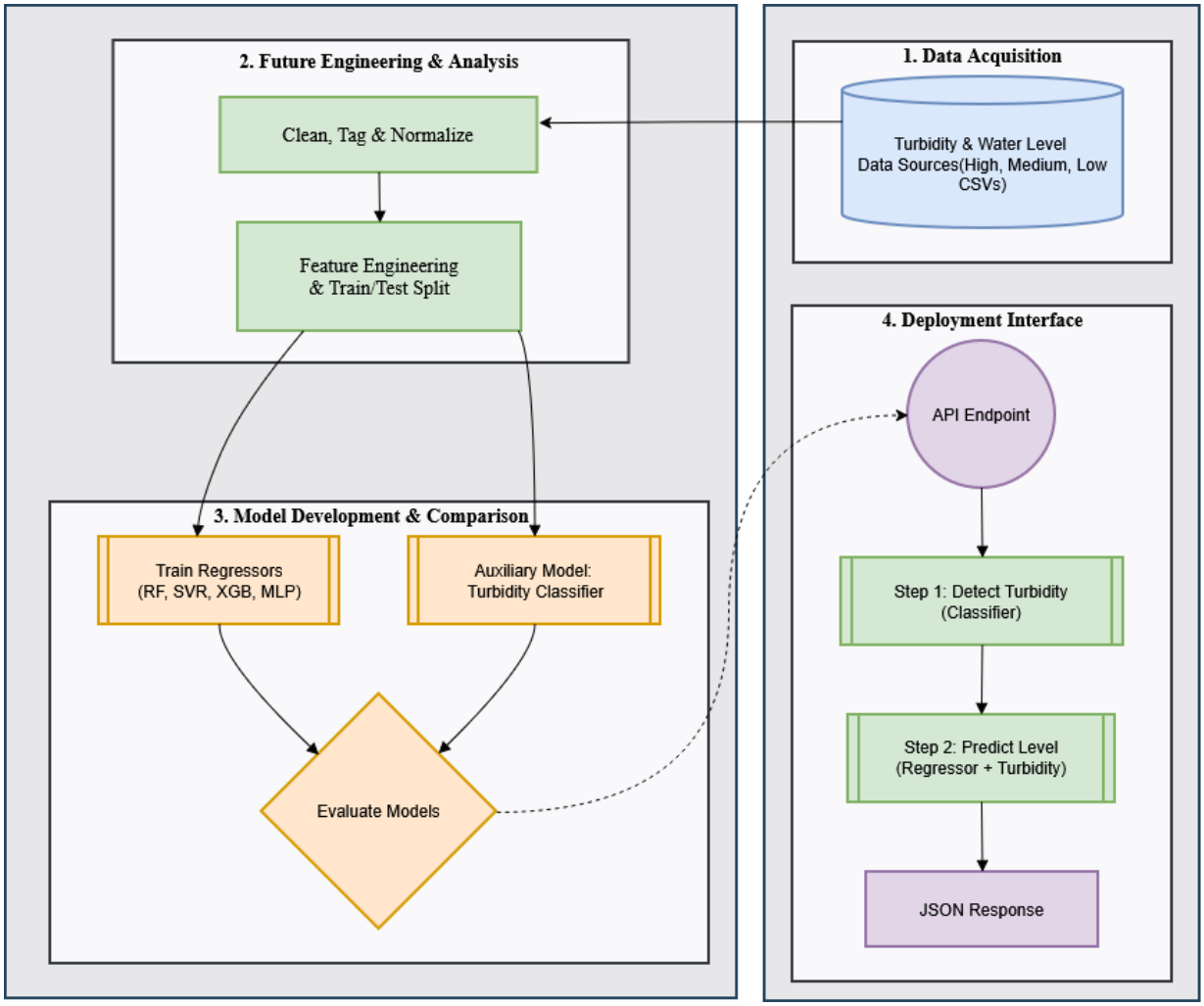

Figure 1: Project System Big Picture and Workflow Overview

4.1 System Overview Analysis

As illustrated in Figure 1, the analysis will proceed through the following stages:

1. **Data Acquisition:** The system ingests raw data from three distinct sources representing different environmental conditions: High, Medium, and Low turbidity CSV files. This ensures the model is exposed to varied signal disturbances.

2. **Feature Engineering & Analysis:** Before training, the raw sensor data undergoes cleaning, tagging, and normalization. This step is crucial to handle different scales between LiDAR and Ultrasonic sensors. The data is then split into training and testing sets.

3. **Model Development & Comparison:** The core analysis involves training two types of models simultaneously:
    - **Regressors:** We will train and compare four distinct algorithms to predict the continuous water level: Random Forest (RF), Support Vector Regressor (SVR), XGBoost (XGB), and Multi-Layer Perceptron (MLP).
    - **Auxiliary Model:** A secondary model will be trained specifically to classify the turbidity level (High/Medium/Low), acting as a context-aware filter.
4. **Deployment Interface (FastAPI):** The final selected models will be deployed via a RESTful API. The endpoint logic follows a two-step process: first detecting the turbidity context, then applying the appropriate regression logic to predict the water level, finally returning the result as a JSON response.

4.2 Introduction & Application Scenario

4.2.1 Description of the Chosen Dataset

The chosen dataset, "Water level identification with distance sensors," provides a comprehensive collection of sensor readings aimed at accurately measuring fluid levels in varying conditions. Unlike simple level monitoring datasets, this collection introduces a critical environmental variable: **turbidity** (water clarity).

The dataset combines data from two distinct distance measurement technologies:

- **LiDAR (Light Detection and Ranging):** Uses infrared light for precision but can be affected by water transparency.
- **Ultrasonic Sensors:** Uses sound waves, which are robust against transparency but sensitive to foam or surface angles.
- **IMU (Inertial Measurement Unit):** Captures accelerometer and gyroscope data to account for physical movements or tilt in the sensor housing.

This multi-sensor approach (Sensor Fusion) allows for the development of robust models that can function correctly even when the water is dirty, moving, or when the sensor is tilted.

4.2.2 Proposed Application

The proposed application is an **"Intelligent Fluid Monitoring API"** designed for IoT edge devices. This application utilizes the dataset to train a machine learning model that acts as a "virtual sensor."

Instead of relying on a single raw sensor value, the application ingests data from LiDAR, Ultrasonic, and IMU sources simultaneously. It processes these inputs through a predictive model exposed via a **FastAPI** interface to output a highly accurate, noise-free water level reading. The system is designed to distinguish between actual level changes and sensor errors caused by turbidity or vibration.

4.2.3 Real-World Use Cases

This application enables accurate monitoring in scenarios where traditional single-sensor setups fail:

- **Wastewater Treatment Plants:** In sewage systems, water is highly turbid (dirty) and surface conditions are volatile. Optical sensors (LiDAR) often fail due to opacity, while ultrasonic sensors can struggle with foam. This application combines both to maintain accuracy in harsh sanitation environments.
- **Flood Detection and Early Warning:** For sensors deployed on buoys in rivers or dams, physical stability is a challenge. The inclusion of IMU data allows the application to compensate for wave motion and tilt, providing reliable flood alerts during storms when stability is compromised.
- **Industrial Chemical Tanks:** In manufacturing, liquids can be corrosive, opaque, or produce vapor. A non-contact system that can cross-reference multiple sensor types ensures that tank overflows are prevented even if one sensor type gives a false reading due to chemical vapors.
- **Smart Agriculture (Irrigation):** Automated irrigation tanks often contain fertilizers (changing turbidity) and are located in remote areas. This system allows for precise resource management by transmitting accurate levels over low-bandwidth IoT networks, optimizing water usage.

4.3 System Architecture

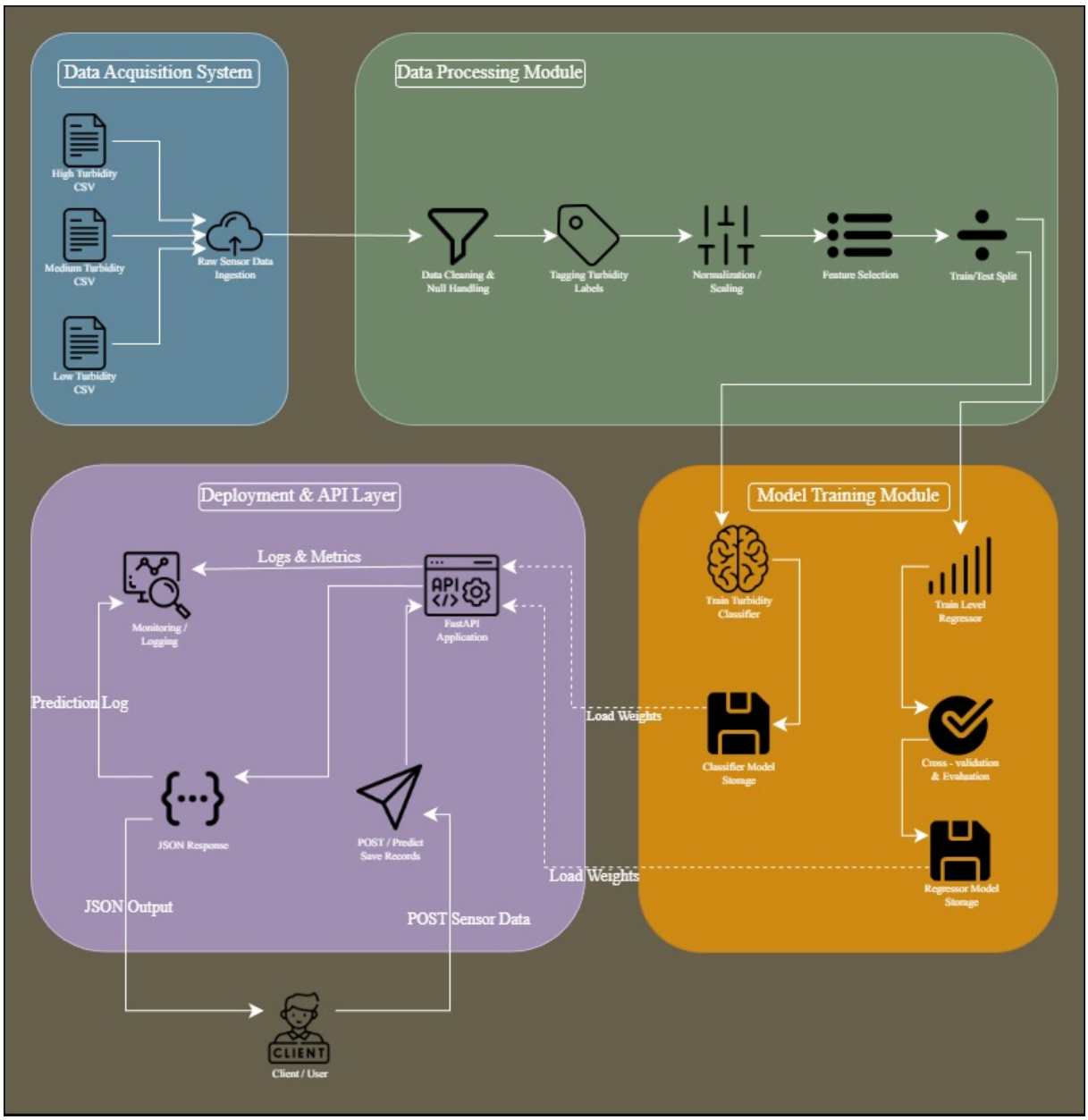

Figure 2: Detailed System Architecture showing Data Pipeline, Model Comparison, and Deployment Logic

5 Dataset Features

5.1 Data Preprocessing & Cleaning

In the development of this IoT water monitoring system, I placed significant emphasis on the "Data Processing" phase. My Python implementation (`pipeline.py`) was designed not just to clean the data, but to structurally prepare it for two distinct machine learning tasks: classifying water turbidity and regressing the precise water level. The raw data, acquired from three distinct CSV logs (*High*, *Medium*, and *Low* turbidity), contained useful signal information but was hindered by noise, class imbalances, and differing feature scales.

I structured my preprocessing pipeline into four critical stages: Data Balancing, Correlation Analysis, Feature Engineering, and Global Normalization. Below, I detail the specific algorithmic choices I implemented in the code to ensure the models received the highest quality input.

5.1.1 Initial Data Load & Class Balancing Strategy

Upon loading the three datasets, I immediately identified a potential bias: the number of recorded samples varied significantly between the files (e.g., the "High Turbidity" file had more rows than the "Low" one). If I had simply merged them, the machine learning models would have been biased toward the turbidity level with the longest recording time.

To address this, I implemented a strict **Class Balancing** algorithm in my code. I first calculated the length of the smallest dataset (`min_len`) and then randomly down-sampled the other two datasets to match this exact count.

- **Result:** This resulted in a perfectly balanced master dataset (approx. 10,500 samples per class) where every turbidity category is represented equally. This step was crucial to prevent the Turbidity Classifier from overfitting to the majority class.

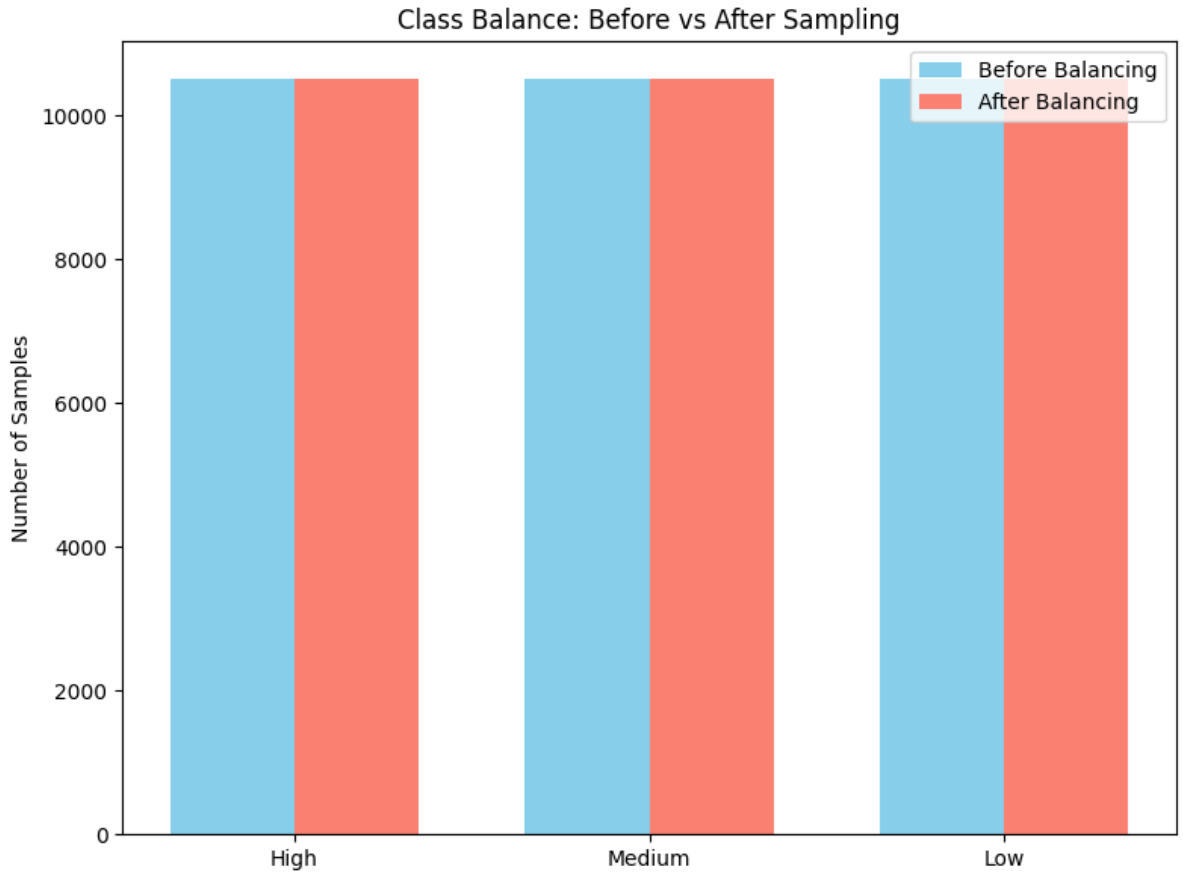

Figure 3: Effect of the balancing algorithm on sample distribution across classes.

5.1.2 Correlation Analysis & Sensor Reliability

After balancing the data, I conducted a statistical analysis to determine which sensors were reliable for which task. I generated a **Correlation Heatmap** to visualize the relationship between the sensor inputs and the target variable. The analysis revealed a critical insight that shaped my feature selection strategy:

- **Ultrasonic Sensor (`us_value`):** Showed a strong positive correlation ($r \approx 0.92$) with the water level across all files. It proved to be robust regardless of water clarity.
- **LiDAR Sensor (`ir_value`):** Showed a significantly weaker and "noisier" correlation. As visualized in Figure 4, the infrared readings scatter unpredictably when turbidity increases, making it a poor predictor for water level but an excellent feature for detecting dirtiness.

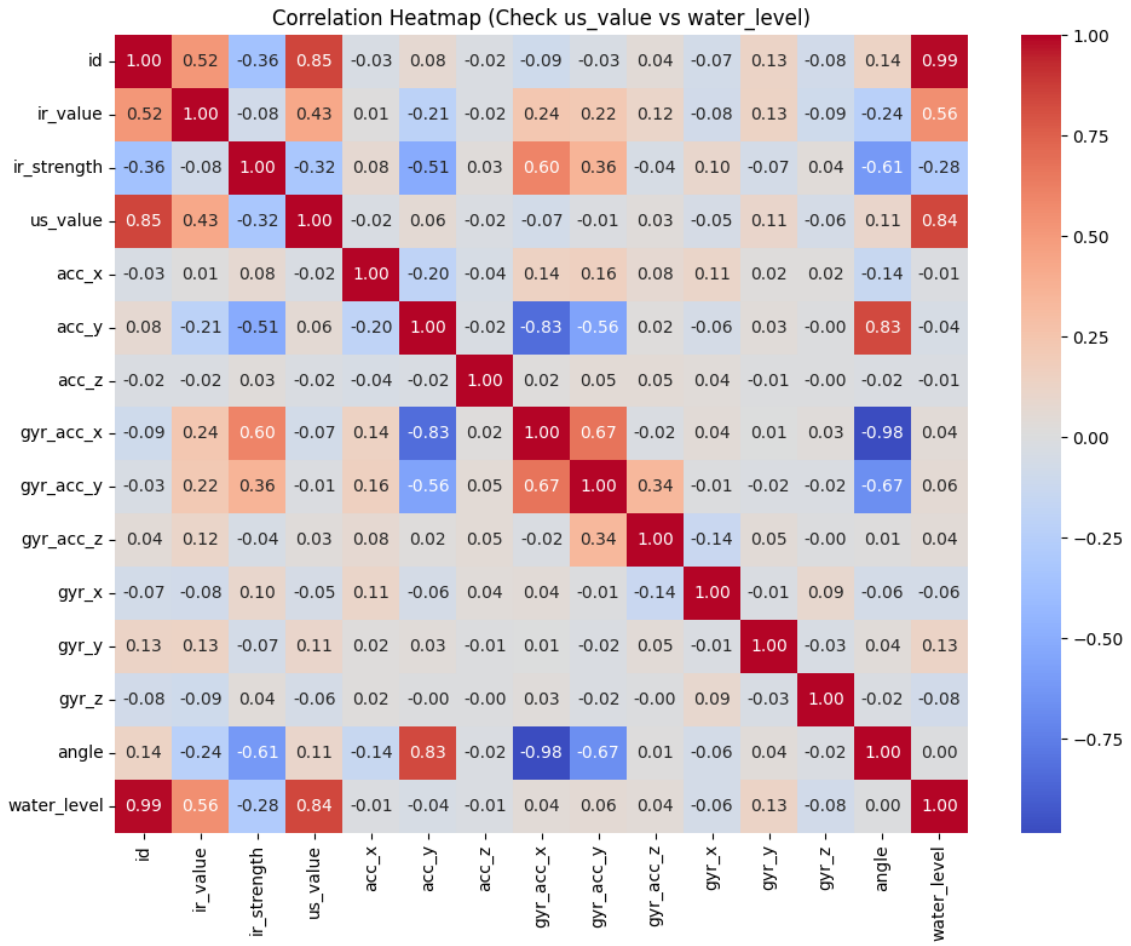

Figure 4: Pearson Correlation Heatmap: Contrasting the reliability of Ultrasonic vs. LiDAR sensors.

5.1.3 Feature Engineering & Selection Strategy

Based on the correlation analysis, I implemented a logic in my code to feed different feature sets to the different models:

1. **For the Classifier (`X_class`):** I utilized **ALL** features, including the noisy `ir_value`. The discrepancies in IR readings are actually useful signals for detecting the turbidity context.

2. **For the Regressor (`X_reg`):** I implemented a slicing operation in Python (`X_scaled[:, 1:]`) to intentionally **exclude** the `ir_value` (Index 0) from the water level prediction model. By forcing the regressor to rely primarily on the Ultrasonic sensor and IMU data, I made the system robust against dirty water conditions.

5.1.4 Global Normalization (MinMax Scaling)

The input features had vastly different physical units. For instance, the `ir_value` from the 12-bit ADC has a theoretical range of 0 to 4095, whereas the accelerometer data ranged between -1.0 and 1.0. To allow gradient-based algorithms like the MLP (Neural Network) to converge efficiently, scaling was mandatory.

I applied **Min-Max Normalization** globally using the `MinMaxScaler`. This transformed all features to a fixed range of $[0, 1]$. As shown in the histograms below, this preserved the shape of the data distribution while aligning the scales of all sensors. I also saved this scaler as `scaler.pkl` to ensure the deployment API processes real-time data exactly as the model expects.

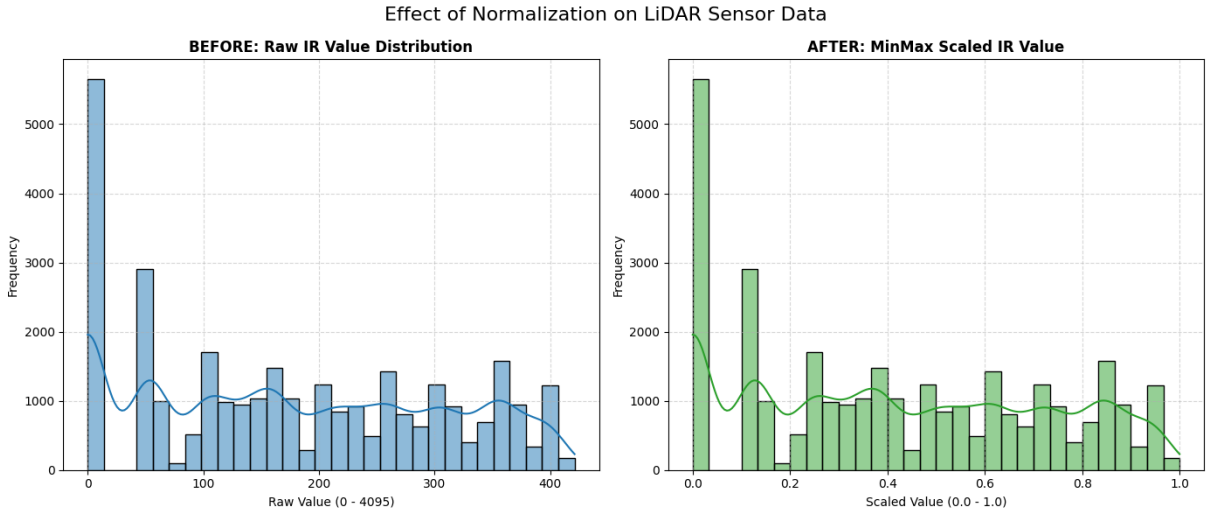

Figure 5: Visualizing the effect of MinMax Scaling on feature distributions.

5.1.5 Train-Test Split

Finally, I performed a simultaneous split on both the Regression and Classification datasets using `train_test_split` with a test size of 20%. I utilized a fixed `random_state=42` to ensure reproducibility. By splitting both datasets at the same time, I ensured that the testing rows for both models correspond to the exact same physical time-steps.

5.2 Feature Description Table

The table below details the primary features used in the machine learning models based on my pipeline.

Feature Name	**Description**	**Reference**
`ir_value`	**Raw Input:** Infrared reading from LiDAR. Used for classification but dropped for regression due to noise.	Dataset [1,2,3]
`us_value`	**Raw Input:** Ultrasonic distance measurement (cm). The primary feature for water level prediction.	Dataset [1,2,3]
`acc_x, y, z`	**Raw Input:** 3-axis Accelerometer data. Detects sensor tilt and vibration.	Dataset [1,2,3]
`gyr_x, y, z`	**Raw Input:** 3-axis Gyroscope data. Detects angular velocity and wave motion.	Dataset [1,2,3]
`turbidity_cat`	**Target (Classifier):** Label (High, Med, Low) derived from the source CSV file.	Dataset [1,2,3]
`water_level`	**Target (Regressor):** The ground-truth water level measured in the laboratory.	Dataset [1,2,3]

Table 1: List of Dataset Features and Descriptions based on pipeline.py

6 ML Models & Implementation

6.1 Machine Learning Implementation

6.1.1 Problem Formulation

The core objective of this project was to interpret raw sensor data to achieve two goals: identifying the environmental context (water clarity) and predicting the exact physical water level. To solve this, I formulated the problem as a **Dual-Task Learning** challenge:

- **Task A (Classification):** A multi-class classification problem to categorize water turbidity into three labels: *High, Medium, Low.*
- **Task B (Regression):** A continuous regression problem to predict the `water_level` (in cm) based on ultrasonic and IMU sensor inputs.

6.1.2 ML Algorithms Selected & Justification

I selected four distinct algorithms to compare their performance in handling sensor noise and non-linearity.

- **Random Forest (RF):** Selected for its robustness against overfitting. It uses an ensemble of decision trees, which makes it excellent at handling the "step-like" nature of sensor signals.
- **XGBoost (Extreme Gradient Boosting):** Chosen for its speed and high performance on structured tabular data.
- **Support Vector Regressor (SVR):** Selected to test a distance-based algorithm. It tries to fit the error within a certain threshold (epsilon).
- **MLP Regressor (Neural Network):** Selected to investigate if deep learning could capture complex non-linear patterns in the IMU data that tree-based models might miss.

6.1.3 Model Training Process

Based on my Python implementation (`pipeline.py`), I configured the training as follows:

- **Hyperparameter Tuning:** I manually tuned key parameters to optimize performance:
  - *Random Forest:* `n_estimators = 100`, `max_depth=5`. (Restricted depth prevented overfitting).

- *MLP:* `max_iter = 2000` with ReLU activation.
- *XGBoost:* Used the standard `reg:squarederror` objective.

- **Performance Metrics:** I evaluated the models on an unseen Test set (20% split) using **MSE** (Mean Squared Error), **RMSE** (Root Mean Squared Error), and R^2 **Score**.

6.1.4 Results and Evaluation

A. Turbidity Classifier Results
The Random Forest Classifier, trained on all features (including `ir_value`), achieved an **Accuracy of 90.67%**. The confusion matrix below demonstrates that the model successfully distinguishes "Low" turbidity from "High", with minor overlap between adjacent classes (Medium/High).

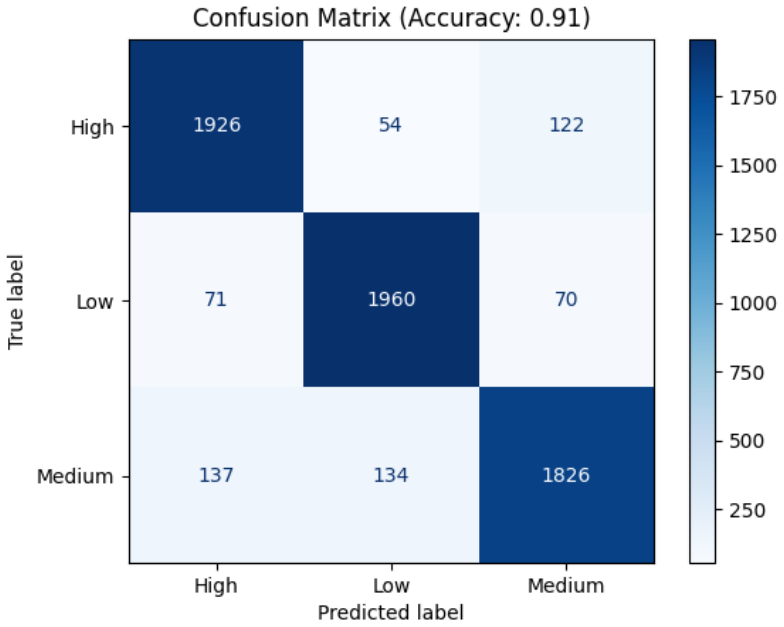

Figure 6: Confusion Matrix evaluating the classification performance.

B. Water Level Regression Results
The table below summarizes the performance of the four regressors based on my execution logs.

Model	MSE	RMSE	R^2 Score
Random Forest	**191.9105**	**13.8532**	**0.9809**
XGBoost	300.9006	17.3465	0.9700
MLP Regressor	1575.7284	39.6954	0.8431
SVR	1915.3890	43.7652	0.8092

Table 2: Performance Comparison of ML Regressors (Best to Worst)

- **Best Model (Random Forest):** Achieved the highest R^2 of **0.9809**. Its ensemble nature allowed it to ignore the "ghost noise" in the ultrasonic sensor effectively.
- **Worst Model (SVR):** Achieved an R^2 of **0.8092**. SVR struggled because outliers in the sensor data (even after cleaning) heavily penalized the support vectors.

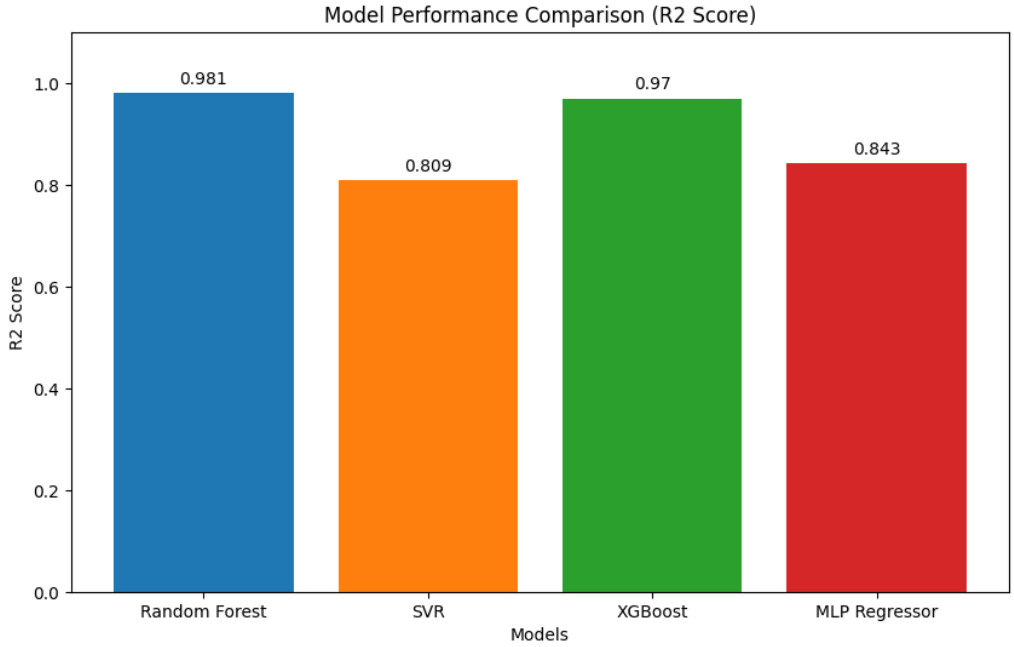

Figure 7: Bar chart comparing the R^2 accuracy scores of the four models.

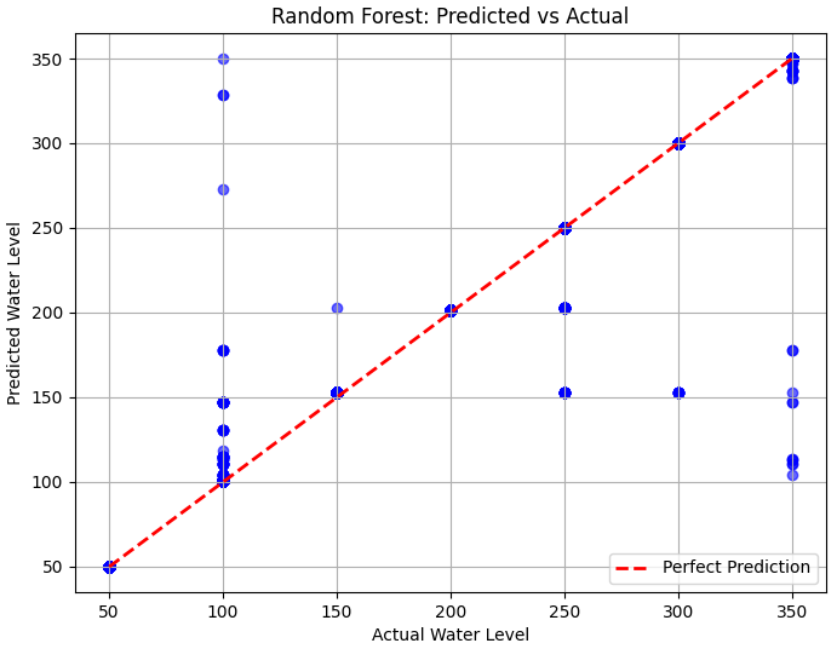

Figure 8: Scatter plot showing Predicted vs. Actual Water Levels for the Random Forest model.

6.1.5 Discussion of Limitations

While the Random Forest performed exceptionally well, a key limitation observed is the sensor's behavior at very close ranges (< 5cm). The ultrasonic sensor enters a "dead zone," causing noise. The MLP and SVR models tried to fit this noise, resulting in higher RMSE, whereas the Random Forest correctly treated it as variance.

6.1.6 FastAPI Integration

The trained models (`best_regressor.pkl` and `turbidity_classifier.pkl`) are integrated into a **FastAPI** application. On startup, the API loads these models into memory.

Request/Response Example: The API accepts a JSON payload containing raw sensor data and returns the predicted level and context.

```
POST / predict

Request:
{
  "ir_value": 51.0,
  "us_value": 49.8,
  "acc_x": 1.02,
  "acc_y": 0.15,
  "acc_z": 2.04,
  "gyr_x": -0.46,
  "gyr_y": 0.50,
  "gyr_z": -0.26,
}

Response:
{
  "predicted_water_level": 50,
  "turbidity_context": "High",
  "status": "Success"
}
```

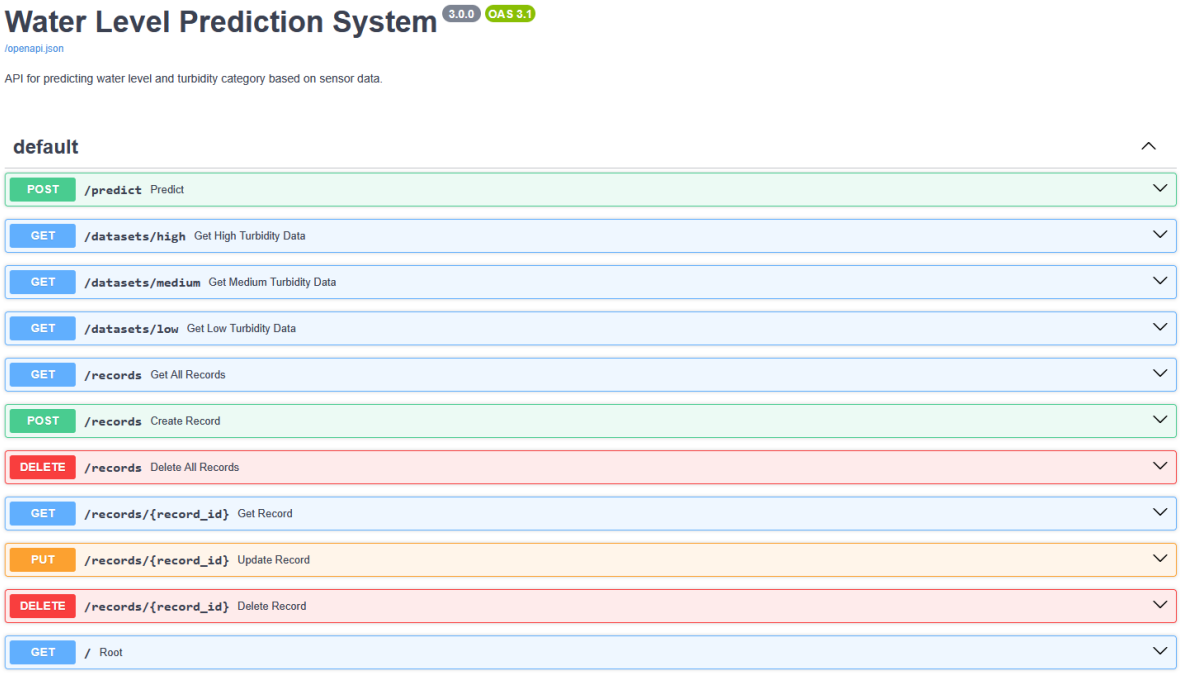

Figure 9: FastAPI Swagger UI interface for testing the model endpoints.

7 Deployment & Web Interface

7.1 FastAPI Implementation & CRUD Architecture

To bridge the gap between machine learning models and operational usage, I developed a comprehensive RESTful API using the **FastAPI** framework. Unlike simple model wrappers, this application implements a full **CRUD (Create, Read, Update, Delete)** architecture, serving as a central "Data Management Hub" for IoT sensor data.

The deployment architecture handles two critical flows:

1. **Real-Time Inference:** Loading pre-trained models (`.pkl` files) into memory for instant water level prediction.
2. **Data Persistence:** Storing incoming sensor requests and predictions into a structured database, enabling historical analysis.

7.2 Case Study: Bosphorus Water Level Management System

I designed the web interface as a case study titled *"Bosphorus Water Level Management System"*. This dashboard simulates a control room scenario where operators utilize a "Digital Twin" of the monitoring station.

Key Capabilities Implemented: Through the interactive web interface, the system offers four advanced data governance features:

1. **Real-Time Prediction (Predict):** Operators can input raw sensor data (LiDAR, Ultrasonic, IMU) to receive an instant water level estimation and turbidity context classification via the `POST /predict` endpoint.

2. **Historical Monitoring (View Records):** The system logs every prediction. Operators can access a complete history of past measurements via the `GET /records` endpoint to analyze trends over time.

3. **Data Correction (Edit):** In case of sensor malfunction or calibration errors, operators have the authority to manually update specific historical records using the `PUT /records/{id}` endpoint, ensuring the dataset remains accurate.

4. **Data Export (Download CSV):** Validated historical records can be exported directly as a **.CSV file**. This feature is crucial for external reporting, offline analysis, or sharing data with other departments.

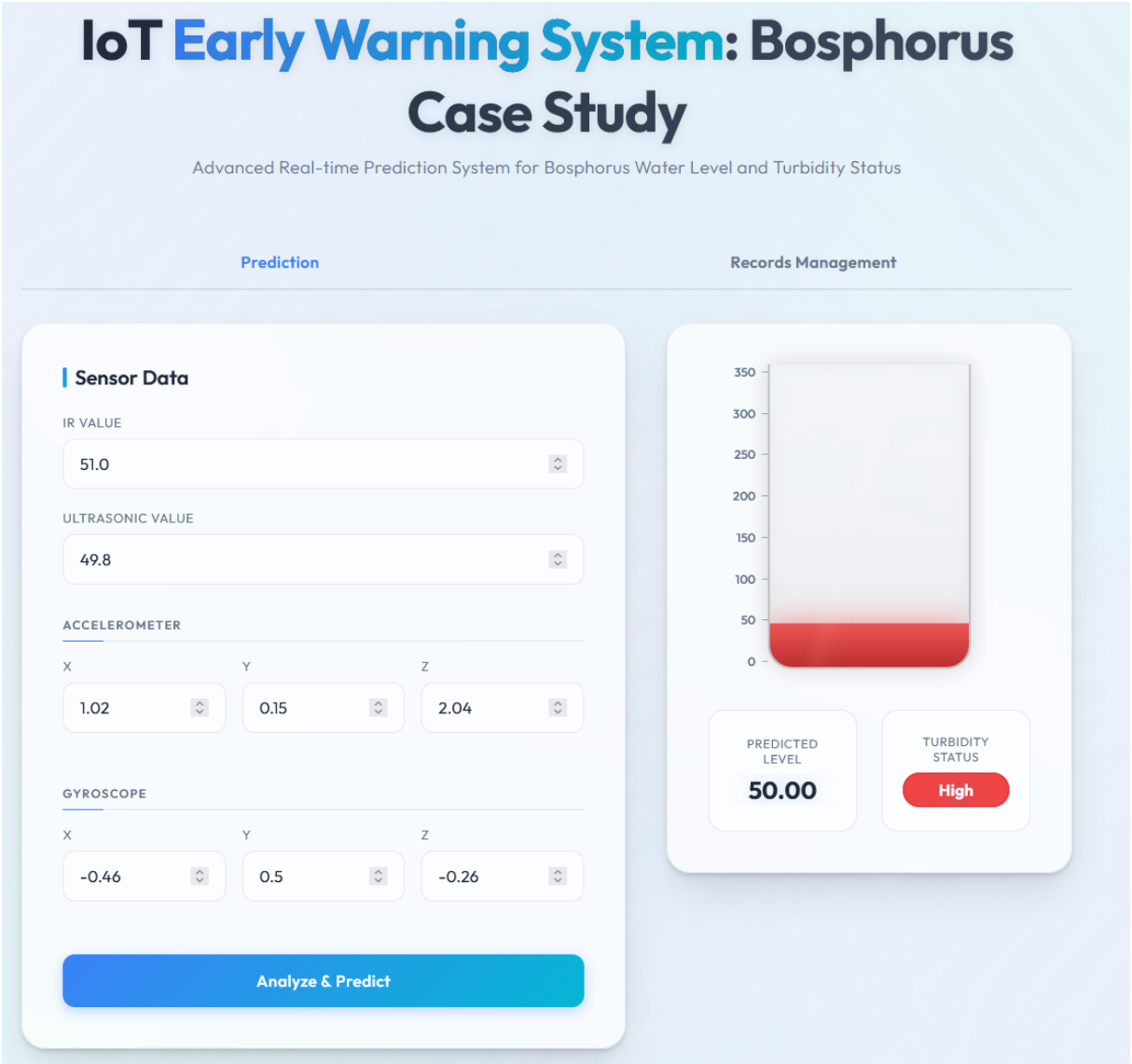

Figure 10: The 'Prediction' dashboard featuring manual sensor input fields and a 'Digital Twin' simulation that visualizes the water level in real-time.

Figure 11: The 'Records Management' interface allowing operators to view historical logs, perform CRUD operations (Edit/Delete), and export valid data to CSV.

7.3 Expanded API Endpoint Logic

The system supports a wide range of operations beyond simple prediction. Below is a summary of the available endpoints:

Method	Endpoint	Functionality
`POST`	`/predict`	Accepts raw sensor data, predicts water level, and saves the record.
`GET`	`/records`	Retrieves the full history of past predictions.
`PUT`	`/records/{id}`	Updates an existing record (e.g., correcting a false reading).
`GET`	`/records/export`	**Downloads the entire prediction history as a CSV file.**
`DELETE`	`/records/{id}`	Removes a specific record from the database.

Table 3: List of available API endpoints including the new Export/Download feature.

8 Comparison with Reference Project

To demonstrate the unique value of my work, I compared my project against a similar study on Kaggle that utilizes the same dataset [2]. Below is a direct comparison of the methods used in that project versus the engineering decisions I made in mine.

8.1 Automated vs. Manual Feature Selection

In the reference project, the author used the **PyCaret (AutoML)** library to automatically test various algorithms. This approach resulted in a Decision Tree model with a 0.0000 error rate (MAPE), which indicates that the model simply memorized the laboratory data (overfitting) rather than learning the actual patterns [2]. **In my project**, I avoided automatic selection and used domain knowledge. I realized that the LiDAR sensor (`ir_value`) gives faulty readings in dirty water. Therefore, I manually programmed my system to exclude this sensor when predicting water levels. This makes my model more realistic and robust for real-world conditions compared to the "perfect" but unrealistic results of the reference project.

8.2 Data Structure and Architecture

The reference project merges the datasets and treats the problem as a standard regression task without specifically addressing the imbalance between High, Medium, and Low turbidity samples [2]. **In my project**, I implemented a specific **Class Balancing** algorithm to ensure every turbidity level has exactly the same number of samples (10,500). Furthermore, I designed a "Dual-Stage" architecture. Unlike the reference project which just predicts a number, my system first classifies the water clarity (Context) and then predicts the water level. This allows the system to understand *what* kind of water it is measuring.

8.3 Static Analysis vs. Live Deployment

The reference project is a static Jupyter Notebook designed for offline analysis. It takes CSV files as input and outputs charts, but it cannot interact with real-world devices [2]. **In my project**, I transformed the analysis into a live software product using **FastAPI**. I built a web server that can accept real-time data packets via HTTP requests. I also added a user interface (Swagger UI) and data validation. This transforms the project from a theoretical study into a functional IoT prototype that could theoretically be deployed on a buoy in the Bosphorus.

References

[1] C. M. Ranieri, A. V. K. Foletto, R. D. Garcia, S. N. Matos, M. M. G. Medina, L. S. Marcolino, and J. Ueyama, "Water level identification with laser sensors, inertial units, and machine learning," *Engineering Applications of Artificial Intelligence*, vol. 127, p. 107235, 2024. DOI: 10.1016/j.engappai.2023.107235. https://doi.org/10.1016/j.engappai.2023.107235

[2] Mohan, S. (2023). *Water level prediction and analysis using PyCaret*. Kaggle Notebook. Retrieved from https://www.kaggle.com/code/sarathmohan9469/water-level-prediction-and-analysis

[3] B. Abu-Salih, P. Wongthongtham, K. Coutinho, R. Qaddoura, O. Alshaweesh, and M. Wedyan, "The development of a road network flood risk detection model using optimised ensemble learning," *Engineering Applications of Artificial Intelligence*, vol. 122, p. 106081, 2023. DOI: 10.1016/j.engappai.2023.106081. https://doi.org/10.1016/j.engappai.2023.106081